Context Engineering for AI Coding Agents

Gunther Brunner

2026-05-15

AI coding agents do real work now — patching production repositories, migrating frameworks, fixing failing tests, answering questions about large codebases — and their success rate is bounded by something less fashionable than raw model capability: *which context they have at hand and how it got there.* A million-token window does not guarantee that the right tokens are present, attended to, version-correct, or trustworthy.

This is a practitioner's field guide to the answer space. It does three things:

1. Catalogs the seven ways context flows into an agent today — vendoring, lazy fetch, repo packing, prose rules, pre-built indices, code overlays, and tool-mediated agentic search — with the mechanism, the winning conditions, the failure modes, and the cost profile of each.
2. Surveys the May 2026 tooling landscape grouped by which strategy each tool implements: 80+ products spanning IDE-native context engines, MCP servers, embeddings/RAG, code search, repo packers, multi-repo systems, and adjacent safety/workflow tools.
3. Offers a fit guide — a decision matrix and a set of worked scenarios — that connects task class, scale, and constraints to a concrete combination of strategies and tools that suits the job.

Empirical evidence (LongCodeBench, CodeRAG-Bench, RULER, structural retrieval studies, distractor sensitivity work) sits underneath the guide as supporting material: it explains *why* the recommendations break the way they do, without becoming the paper's central argument. Standards and interoperability (MCP, AGENTS.md, Skills, llms.txt) are surveyed briefly because they determine which combinations are portable across agents.

The guide is shaped by work on *ingraft*, an open-source CLI for making repository context explicit for coding agents. Its flagship route is version-matched upstream source in the workspace, but the broader problem is routing: when should context be vendored, linked, fetched, packed, searched, or exposed through a tool? ingraft is treated here as one implementation slice rather than the protagonist. The reader's question is the protagonist: "what should I use for this job?"

Table of contents

1 Introduction

AI coding agents are now useful enough that their failures matter. Developers ask them to patch production repositories, migrate frameworks, inspect failing tests, answer questions about large codebases, and integrate unfamiliar libraries. In these settings, the bottleneck is rarely "no relevant information exists." The bottleneck is context allocation: which information should enter the agent's working set, in which representation, through which trust boundary, at what point in the loop, and under what budget.

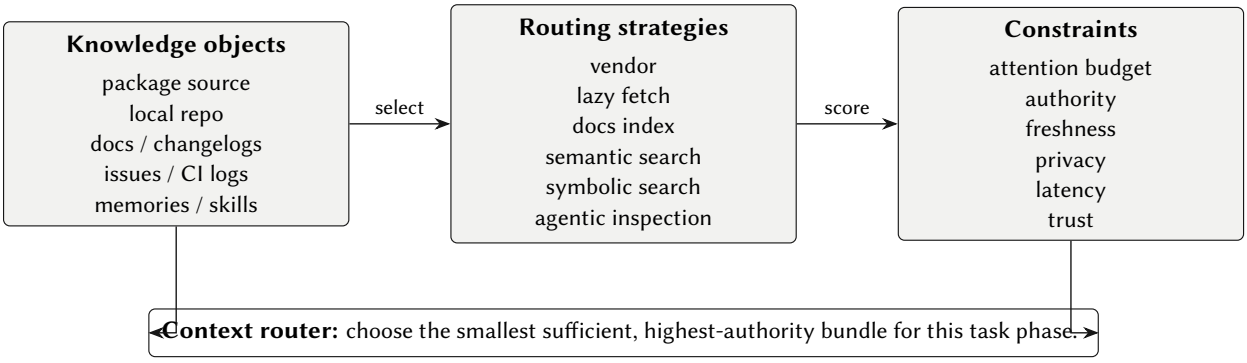

Figure 1: Context routing at a glance. The paper's thesis is not that one context source wins; it is that coding agents need an explicit allocation policy over objects, routes, and constraints.

This guide treats that question as a practical problem with practical answers, not a research thesis. Seven distinct ways of moving context into an agent are in active use today, dozens of products implement each

one, and most real tasks need a *combination* of two or three. The goal of the guide is to make those choices legible to the person who has to make them.

1.1 What this guide covers

The body answers a reader's questions in order:

- **§2, The strategies** — the seven ways context flows into an agent today. For each: how it works, where it wins, where it loses, what it costs.
- **§3, The tools** — the May 2026 product catalog grouped by which strategy each implements.
- **§4, The fit guide** — a decision matrix and seven worked scenarios that connect task class, scale, and constraints to a concrete combination of strategies and tools.
- **§5, The evidence** — the research papers and benchmarks that explain *why* the recommendations break the way they do.
- **§6, How to think about routing** — a framework (placement, representation, selector, timing, constraints) for reasoning about a route when the decision matrix does not cover the case.
- **§7, Standards and interop** — the ecosystem-wide protocols (MCP, AGENTS.md, Skills, llms.txt) that determine which combinations are portable across agents.
- **§8, Where ingraft fits** — one slot in the vendoring strategy.

The guide is a dated snapshot. Most tool and standards claims were checked against public documentation, repositories, specifications, or vendor posts available during March–May 2026. Tool counts and product features age fast.

The timing matters because coding agents are no longer only IDE chat assistants. Public product documentation now describes agents that run asynchronously in cloud worktrees, PR queues, desktop apps, mobile handoffs, and parallel review fleets (OpenAI 2026a; OpenAI 2026b; Anthropic 2026a; GitHub 2026a; GitHub 2026b; Google 2025). In that world, the repository becomes the most durable context contract: project files, source snapshots, route metadata, and agent instructions survive handoffs more reliably than conversational memory.

The immediate product trigger was Maxwell Brown's Effect post on using git subtrees for coding-agent context (Brown 2026): "coding agents rely on patterns and examples, not descriptions; give your agent real library code to learn from." That claim is intentionally narrower than this guide. It does not settle the general problem; it points at it: if subtree is the right route for Effect, what decides when subtree is right, when docs are better, and when neither should be loaded? The field-guide answer is: it depends on the task, the scale, and the constraints — and the rest of the document is the dependency table.

1.2 Vocabulary

Six terms recur throughout. They are defined once here.

- **Context** — any information made available to an agent during a task: prompt tokens, files it can read, tool results, memory, indexes, skills, instructions, logs, external service responses.
- **Context object** — a knowledge unit that might be relevant: a package, file, repository, issue, stack trace, lockfile entry, API document, design note, generated summary, skill.

- **Strategy** — a way of exposing a context object to the agent. The seven strategies in §2 are the catalog of options.
- **Bundle** — the concrete material exposed to the agent for a given task: selected files, docs, snippets, index hits, rules, tool affordances.
- **Authority** — which source should win for a specific question: the lockfile for installed version, official docs for public API intent, source and tests for behavior, local code for project convention, human instructions for constraints.
- **Provenance** — where a bundle came from, which version it represents, when it was fetched, how trustworthy it is, and what it is intended to be used for.

A seventh term, **context router**, appears in §6 as the name for the explicit policy that selects among strategies. It is one organizing lens, not a prescription. The guide works whether or not you adopt the term.

2 The Strategies

Seven distinct strategies move context from somewhere outside the agent into the agent's working set. Almost every product on the market implements one or more of them; the difference between products is which combination they bake in. This chapter is the catalog: for each strategy, the mechanism, where it wins, where it fails, and the cost profile.

A reader who only reads this chapter should be able to recognize, given any tool, which strategy or combination of strategies the tool implements — and therefore which jobs the tool was built to do.

2.1 Strategy 1 — Vendoring (source becomes part of the workspace)

Mechanism. Upstream source is committed or mounted directly inside the project, typically under a reference directory like `vendor/`, marked read-only for humans and visible to the agent. Implementations range from raw `git subtree`, `git submodule`, and `git clone` to wrappers like ingraft, Vendir, and Carvel that automate update, ignore-pattern injection, and provenance.

Wins when: a single package is a load-bearing idiom source for the project (Effect, Inertia.js, complex internal frameworks); the agent needs concrete patterns and tests, not prose descriptions; the source is navigable and version-matched; privacy rules out cloud indexing; offline operation matters.

Loses when: the package is huge, generated, or full of distractors (Chromium-scale repos); the project will get tens of dependencies vendored and the repo bloats; the codebase needs public-API *intent* more than implementation behavior (docs win there); the team needs to upgrade often and the wrapper does not automate updates.

Cost profile. High upfront setup and ongoing maintenance. Near-zero per-request latency and per-request token cost (the agent just reads files). High determinism. Privacy is local-by-default.

2.2 Strategy 2 — Lazy fetch (on-demand pull into a side cache)

Mechanism. The agent asks for a package or doc, the tool resolves it (npm, PyPI, crates.io, etc.), and the result lands in a side cache (`~/.opensrc`, `node_modules_cache`). The cache is read like a local filesystem, then either retained or pruned.

Wins when: most packages are used occasionally and not worth vendoring; the project's dependency surface is large but shallow; one or two packages may need deeper inspection during debugging.

Loses when: latency on the first cache miss matters; the project needs guaranteed offline operation; package registries are themselves a privacy or supply-chain risk.

Cost profile. Very low upfront cost, low maintenance. Latency spikes on cache miss; otherwise local-fast. Token cost low because only needed files are loaded.

2.3 Strategy 3 — Repo packing (snapshot the project into a single artifact)

Mechanism. A tool (Repomix, code2prompt, gitingest, files-to-prompt, yek, ai-digest, gpt-repository-loader) walks the repository, applies ignore rules, and produces one large markdown or XML blob that the user pastes (or the agent loads) as a single prompt attachment.

Wins when: the agent has no filesystem or shell tools (web chat interfaces, hosted assistants without code access); the task is a one-shot Q&A about a repo ("what does this project do?"); the team wants a reproducible artifact tied to a specific commit.

Loses when: the repo is larger than the pack will compress into; the model has limited effective context length; the task requires multi-turn agent loops (the packed snapshot goes stale immediately on any edit); position-sensitive attention biases hurt the buried files.

Cost profile. Cheap per snapshot, expensive per token in context. Zero determinism over time (one snapshot is deterministic; a re-pack on a later commit is not).

2.4 Strategy 4 — Instruct (prose rules and skills)

Mechanism. Project-level instruction files (`AGENTS.md`, `CLAUDE.md`, `GEMINI.md`, `.cursor/rules`, Anthropic Agent Skills) tell the agent how this codebase wants to be worked on. Skills are progressive-disclosure units: a short `SKILL.md` that the agent reads first, with deeper procedural files the agent loads only when needed.

Wins when: the knowledge is procedural ("always run the linter before committing"), conventional ("we use snake_case in Python, camelCase in TypeScript"), or stable ("never modify `vendor/`"); the team wants a single canonical place for "things every agent should know."

Loses when: the rules are wording-sensitive (Vercel's Next.js evals: skill-based docs lookup only worked with explicit instruction; default skill triggering was ignored); the rules drift away from the code as the project evolves; the model decides not to follow them.

Cost profile. Almost free to author and maintain. Always-on prompt overhead is small if rules are concise; large if rules are bloated.

2.5 Strategy 5 — Pre-built index (semantic, symbolic, or graph)

Mechanism. A background process indexes the codebase (embeddings, BM25, AST, LSP symbols, code graph) and exposes a search interface to the agent. The agent searches the index instead of reading files directly. Implementations: Cursor's built-in index, Sourcegraph's code graph (SCIP), Augment's enterprise index, Cody, Sourcebot, Bloop, Tabnine, Glean, Continue.

Wins when: the codebase is large enough that grep is slow or the search query is semantic rather than lexical; cross-file relationships matter; the same agent will run many tasks against the same codebase and the index is amortized.

Loses when: the codebase changes rapidly and the index drifts; the index is hosted in the cloud and the codebase is sensitive; setup cost is large for one-shot use.

Cost profile. High upfront index build and ongoing re-indexing. Low per-request latency once warm. Token cost low (only hits are loaded).

2.6 Strategy 6 — Annotate (overlays from external systems)

Mechanism. Context that does not live in the repo — code review threads, Jira issues, design docs, Slack discussions, customer feedback — is exposed as overlays linked to lines, symbols, or files. OpenCtx (now relatively dormant), NLWeb, and Sourcegraph's code-graph context retrieval are the canonical patterns.

Wins when: the missing context is institutional knowledge that explains *why* code looks the way it does; the team has discipline about putting decisions in linked systems.

Loses when: the overlay systems are stale; ACLs prevent the agent from reading them; the institutional context never made it into a queryable system.

Cost profile. Low per-request once integrated; integration is itself the cost.

2.7 Strategy 7 — Tool-mediated agentic search

Mechanism. The agent has bash, grep, file reads, edit, and sometimes web tools, and the model decides what to inspect each turn. There is no pre-built index — the model navigates the filesystem in real time, often with reasoning between each tool call. Claude Code, Cline, OpenHands, SWE-agent, aider, and many MCP-enabled CLIs work this way.

Wins when: local files are available; the codebase changes faster than an index can re-build; the model is strong enough to plan multi-step searches; freshness matters (uncommitted changes, recent edits).

Loses when: the search loop drags on (latency and token cost blow up); the task requires cross-file structural reasoning that grep cannot produce; the model spirals through file reads without converging.

Cost profile. Zero upfront cost (you already have a shell). High latency and high token cost per task. Lowest determinism: two runs can search different files in different orders.

2.8 Strategies compose

In real deployments the seven rarely run alone. A typical Claude Code setup combines instruct (CLAUDE.md), tool-mediated (bash + grep + read), skill (progressive disclosure), and lazy fetch via MCP. Cursor combines index (default), retrieve (`@docs`), and instruct (`.cursor/rules`). Codex and Copilot increasingly add cloud, PR, and app surfaces around the same repository contract. Effect's git-subtree recommendation is pure vendor leveraging the agent's tool-mediated capability over the embedded source.

Figure 12 compares the seven strategies on four cost axes (per-request latency, token cost, upfront cost, non-determinism), so a strategy can be picked by which costs the task can afford.

3 The Tools

What follows is a May 2026 snapshot of the products and projects that implement each strategy. Tool counts and product features age fast; treat this as a navigation aid, not a static reference. The full extended snapshot lives in Appendix 10.

3.1 IDE-native context engines

These are the products developers spend most hours inside. Each one bakes in a default combination of strategies.

Table 1: IDE-native context engines and the strategies they implement.

Tool	Default strategy mix	Where it fits best
Cursor	Index + instruct (`.cursor/rules`) + retrieve (`@docs`)	Day-to-day work in a large, stable codebase that benefits from amortized indexing
OpenAI Codex	Tool-mediated agentic + cloud/app worktrees + skills/automations	Parallel local/cloud work where repo files, tests, and review state must survive handoffs
Claude Code	Tool-mediated agentic + instruct (CLAUDE.md) + skills + web/cloud review	High-context single-task work and multi-agent review where freshness and exploration matter more than pre-built indices
GitHub Copilot	Index + tool-mediated + coding agent + Copilot CLI	Teams already deep in the GitHub ecosystem; PR, issue delegation, mobile, and IDE workflows
Google Jules	Async cloud coding agent over GitHub repos	Background fixes and upgrades where a managed environment can own the task loop
Augment Code	Enterprise index + tool-mediated; 100M+ LOC repos; on-prem option	Highly regulated or massive proprietary codebases
Sourcegraph Amp	Code graph (SCIP) + tool-mediated; multi-repo	Cross-repo enterprise navigation
JetBrains AI / Junie	Index + tool-mediated, IDE-native	Teams committed to JetBrains IDEs

Tool	Default strategy mix	Where it fits best
Cline / Roo Code / Kilo Code	Tool-mediated agentic via MCP	VS Code users who want bring-your-own-model
aider	Tool-mediated agentic + tree-sitter repo map + git-native commits	Terminal-first developers who want repo-aware AI pair programming
Continue	Open-source IDE assistant; pluggable	Organizations that need fully auditable open-source tooling
Devin / Factory Droid / OpenHands	Autonomous agent with managed env	Long-horizon autonomous tasks where the agent runs unattended
Warp (Agent Mode)	Tool-mediated + WarpGrep RL retriever	Terminal-centric workflows; large local repos
Amazon Q Developer / Kiro	Index + instruct, on AWS Bedrock	Teams already standardized on AWS

3.2 MCP servers and standalone fetchers

MCP servers expose context as tools the agent can call. The 2026 MCP ecosystem includes 10,000+ public servers; the table covers the ones most relevant to coding agents.

Table 2: MCP servers and standalone context fetchers.

Server / tool	Strategy implemented	Where it fits best
Context7	Pre-built per-library docs index over 33K+ libraries	Public-API & migration tasks; version-matched docs
filesystem MCP	Tool-mediated access to a sandboxed local FS	Universal building block for agentic search
GitHub MCP / GitLab MCP	Tool-mediated access to issues, PRs, code	Tasks that need repo metadata beyond the working tree
DeepWiki MCP	Pre-built docs/wiki index from public repos	"What does library X do?" research
mgrep / Serena MCP	Symbol/AST/LSP retrieval (Strategy 5, structural variant)	Cross-file structural questions; languages with LSP support
Mem0 / Letta / Mnemos / Pieces LTM-2	Long-term memory store (Strategy 6, annotate-style)	Multi-session work that needs continuity
OpenSrc	Lazy package fetcher into `~/.opensrc`	On-demand source inspection without vendoring
Exa MCP / Tavily MCP / Perplexity MCP	Web search exposed as a tool	Tasks that need fresh web evidence beyond the repo

3.3 Embeddings, RAG, and code search components

These are substrates inside other tools more often than end-user products. They get picked when teams build their own retrieval stack.

- **Code-trained embeddings:** voyage-code-3, Jina code embeddings, mxbai-embed-large, CodeBERT, GraphCodeBERT, UniXcoder, CodeT5+, CodeSage. Pick by language coverage and corpus size; benchmark on your own data because cross-domain transfer is uneven.
- **Vector databases:** ChromaDB, Qdrant, Pinecone, Weaviate, Milvus, pgvector. The pick rarely matters at small scale; at large scale, focus on incremental updates and ACL support.
- **Code search engines:** Sourcebot, Zoekt, Livegrep, Hound, ast-grep, Comby, Bloop, Greptile, Tabby. Pick AST/structural over flat embedding wherever language tooling permits (Pillar C evidence in §5).
- **Indexing pipelines:** CocoIndex, Indexify, FreshStack-style fresh document retrieval. Pick when freshness or incremental updates are first-class.

3.4 Repo packaging and snapshots

Repomix, code2prompt, gitingest, files-to-prompt, yek, ai-digest, gpt-repository-loader. Each walks the repo, applies ignore rules, and emits a single artifact. Pick by language support, default ignore behavior, and whether you need XML, markdown, or JSON output. None of these is wrong; they differ in defaults, not in capability.

3.5 Multi-repo and org context

RepoSwarm, Backstage TechDocs, Roadie RAG, Glean, Sourcegraph, DeepWiki, Unblocked. The defining constraint here is ACLs and multi-repo scale — ordinary IDE engines do not handle either well. Pick when "the right context is in another repo" is a routine problem.

3.6 Vendoring wrappers

ingraft, Vendir (Carvel), git subtree directly, git submodule directly, git clone with ignored directory. These differ in how they handle updates, ignore-pattern injection, and provenance, not in what they store. §8 covers ingraft's specific slot.

3.7 Adjacent safety, review, and workflow tools

Qodo Merge, CodeRabbit, Greptile review mode, Sourcery review, GitGuardian ggshield AI Hook, MCP-omnisearch, GPTScript, Jujutsu, agentjj. These are not primary context providers but they consume, filter, or audit the context routed to agents during PR review, secret detection, search aggregation, or VCS workflows.

4 The Fit Guide

This is the chapter the rest of the guide exists for: given a concrete task and a set of constraints, which combination of strategies and tools should the reader reach for? The decision matrix (Table 3) maps task class to recommended primary and supporting strategies, and the worked scenarios that follow show each combination playing out in practice.

Table 3: Decision matrix — task class to recommended strategy combination.

Task class	Primary strategy	Supporting strategies	Representative tools
Learn an idiom-heavy framework	Vendor	Tool-mediated agentic + instruct	ingraft / git subtree + Claude Code
Major version migration	Docs index (instruct)	Lazy fetch + tool-mediated	Context7 / AGENTS.md docs index + Claude Code or Cursor
Onboard to large unfamiliar codebase	Pre-built index	Tool-mediated + repo map	Cursor or Sourcegraph Amp + aider repo-map
Cross-repo enterprise refactor	Multi-repo index (Strategy 5+6)	Tool-mediated	Sourcegraph Amp / Glean / Augment
Debug a flaky / failing test	Tool-mediated agentic	Lazy fetch for package source	Claude Code / Cline / aider + OpenSrc
One-shot Q&A on an unfamiliar repo	Repo pack	—	Repomix / gitingest
Security review on a sensitive codebase	On-prem index + instruct	Tool-mediated, agentic search disabled	Augment / Tabnine on-prem; strict trust policy
Test repair after a refactor	Tool-mediated agentic	Symbol/AST index	Claude Code + Serena MCP / mgrep

4.1 Scenario 1 — Learn an idiom-heavy framework

Situation. The team starts a TypeScript service that depends heavily on Effect, a framework whose human-facing docs are precise but deliberately abstract; the patterns live in the source.

Recommended bundle. Vendor (Strategy 1) the Effect source into `vendor/effect`, mark it read-only via `AGENTS.md` (Strategy 4), and use Claude Code or Cursor (Strategy 7, tool-mediated) over the embedded source. The agent finds idioms by grep + read on real code and tests. Pulled docs (Strategy 2 lazy fetch) cover migration questions when the source view is not enough.

Why this wins. Source-over-docs evidence (Brown 2026) is strongest for idiom-heavy frameworks. The vendored copy gives the agent navigable, version-matched patterns with zero per-request latency or token cost. The `AGENTS.md` rule keeps humans from editing vendored files by mistake.

Antipatterns. Pasting Effect docs into the prompt every turn; indexing a third-party library that changes weekly; relying on the model's pre-training memory of Effect 2.x while the project uses 3.x.

4.2 Scenario 2 — Major version migration (Next.js 15 → 16)

Situation. A team must upgrade a Next.js 15 app to Next.js 16. Hundreds of small API changes, several breaking ones, and the existing code uses deprecated hooks.

Recommended bundle. Always-on docs index (Strategy 4, the Vercel pattern): a compressed `AGENTS.md` that points at a local `.next-docs/` directory with version-matched migration docs. Pair with Strategy 5 (symbol/AST index of the local codebase) so the agent can locate every deprecated API call by symbol, not by substring. Tool-mediated agentic (Strategy 7) does the edits.

Why this wins. Vercel's own Next.js eval suite (Figure 2): the `AGENTS.md` docs-index pattern scored 100% on their hardened eval, versus 53% baseline and 79% with skill plus explicit instruction (Gao 2026a). Migration tasks are public-API tasks; docs are authority, source is not.

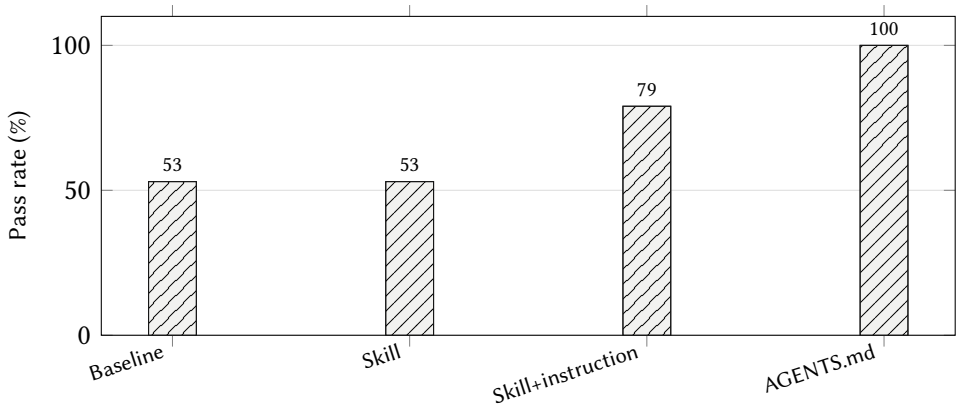

Figure 2: Vercel's reported Next.js agent eval results. The lesson for routing is selector and timing: an always-on compact docs index outperformed an on-demand skill in this suite.

Antipatterns. Vendoring Next.js source (huge, generated, distractor-heavy); web-searching every API change (stale, no version guarantee); relying on the model's pre-training memory of an older Next.js.

4.3 Scenario 3 — Onboarding to a large unfamiliar codebase

Situation. A new engineer joins a 2M-LOC monorepo.

Recommended bundle. Pre-built index (Strategy 5, semantic + structural) for orientation; tool-mediated agentic (Strategy 7) for deep dives; aider-style tree-sitter repo map (Strategy 3, pack-compressed) for "which files matter for X". Instruct (Strategy 4) project rules via `AGENTS.md` or `CLAUDE.md` so the agent inherits team conventions.

Why this wins. 2M LOC is past the break-even point where index maintenance pays off: indexing wins over agentic search when repository size, query frequency, and cross-file complexity are large enough to amortize the build and re-indexing cost ($C_{\text{toolCalls}} + C_{\text{tokens}} + C_{\text{latency}} > C_{\text{indexBuild}} + C_{\text{indexDrift}} + C_{\text{retrievalErrors}}$). Repo maps are the right orientation tool because exact code is too long for orientation and architecture diagrams drift.

Antipatterns. Repo-packing 2M LOC into a single prompt; forcing agentic search to grep across the entire monorepo on every turn; skipping `AGENTS.md` and re-learning conventions task by task.

4.4 Scenario 4 — Cross-repo enterprise refactor

Situation. A platform team must rename a deprecated API used across 40 internal repos.

Recommended bundle. Multi-repo managed index (Strategy 5 + Strategy 6 annotate, Glean / Sourcegraph / Augment) with ACL-aware search. Tool-mediated agentic (Strategy 7) per repo to do the edits. Instruct (Strategy 4) the policy via `AGENTS.md` in each repo so the agent picks up project-specific overrides.

Why this wins. Org context requires ACLs, multi-repo scale, and governance that IDE engines do not handle well.

Antipatterns. Doing it from a single IDE with no cross-repo visibility; using a public cloud index for proprietary code.

4.5 Scenario 5 — Debug a flaky / failing test

Situation. Test `TestPaymentRetry` fails intermittently on CI.

Recommended bundle. Tool-mediated agentic (Strategy 7) is the primary: the agent reads logs, greps the test, follows call chains, inspects the package source if needed. Lazy fetch (Strategy 2) provides on-demand package source for any third-party retry library. No pre-built index needed.

Why this wins. Debugging is exploratory by nature; pre-built indexes go stale faster than uncommitted changes; the cost of agentic search is amortized over a short interactive session, not 40 production tasks.

Antipatterns. Indexing for a one-off debug; vendoring a dependency just to read three functions; repo-packing the entire codebase when the bug is in two files.

4.6 Scenario 6 — One-shot Q&A on an unfamiliar repo

Situation. An engineer wants to ask "what does this open-source repo do, and how does its auth module work?" from a web chat interface with no shell tools.

Recommended bundle. Repo pack (Strategy 3) with Repomix or gitingest, paste into the chat.

Why this wins. Repo packs are the right tool when (a) the agent has no filesystem and (b) the task is one-shot. The downsides (staleness, position bias) do not matter for a single Q&A.

Antipatterns. Setting up an MCP server, an index, and a vendoring pipeline for a question that takes 60 seconds.

4.7 Scenario 7 — Security-sensitive codebase

Situation. A regulated team works on financial-services code where source must not leave the corporate network.

Recommended bundle. On-prem index (Strategy 5; Augment VPC / Tabnine air-gapped / self-hosted Sourcegraph) + instruct (Strategy 4) with strict trust policy that disables web fetch and external MCP servers. Tool-mediated agentic (Strategy 7) only against local files.

Why this wins. Privacy constraints rule out cloud retrieval. The on-prem index keeps the productivity gain without the data egress. Strict trust policy on `AGENTS.md` closes the prompt-injection surface.

Antipatterns. Sending repo content through any third-party LLM that retains training data; using public MCP servers without auditing them; relying on developer discipline alone to keep secrets out of prompts.

5 The Evidence — Why These Choices Matter

The recommendations in §2–§4 are shaped by a decade of research on long-context models, retrieval, and agent scaffolding. This chapter is the supporting layer: the studies and benchmarks that explain *why* the strategies break the way they do. A reader who trusts the recommendations can skim it; a reader who wants to argue with them should not.

The original section title was "Why Context Allocation Is Hard," and that framing still organizes the material into four pillars: long context degrades, retrieval is conditional, structure beats flat, and distractors actively hurt.

5.1 Context windows are not attention guarantees

Long windows increase storage, not reliable computation over every token. N. F. Liu et al. (2024) established the now-canonical positional failure: models perform best when relevant information is near the beginning or end of the context and degrade when evidence is buried in the middle. Hsieh et al. (2024) showed that advertised context length can overstate effective context length, especially on multi-hop synthetic tasks.

LongCodeBench extends the problem to realistic code comprehension and repair (Rando et al. 2025). It evaluates LongCodeQA and LongSWE-Bench across 32K, 64K, 128K, 256K, 512K, and 1M context brackets where supported. The two tasks have to be reported separately — collapsing them into one "LongCodeBench accuracy" number is the most common misreading of the paper, and it obscures the result that matters.

> On **LongSWE-Bench** (executable bug-fix pass rate on real GitHub issues), Claude 3.5 Sonnet solved 29% of issues at 32K and 3% at 256K; it was not evaluated at 512K or 1M. Gemini 2.5 Pro held 22–25% through 256K and only collapsed at 512K/1M (12%, 7%). GPT-4.1 sat near 1% at every bracket. Qwen2.5, Llama 3.1, and Llama 4 Scout each solved zero issues at every bracket. On **LongCodeQA** (multiple-choice comprehension), Qwen2.5 peaked at 70.2% at 512K and fell to 40.0% at 1M, while GPT-4.1 and Gemini 2.5 Pro showed no comparable cliff.

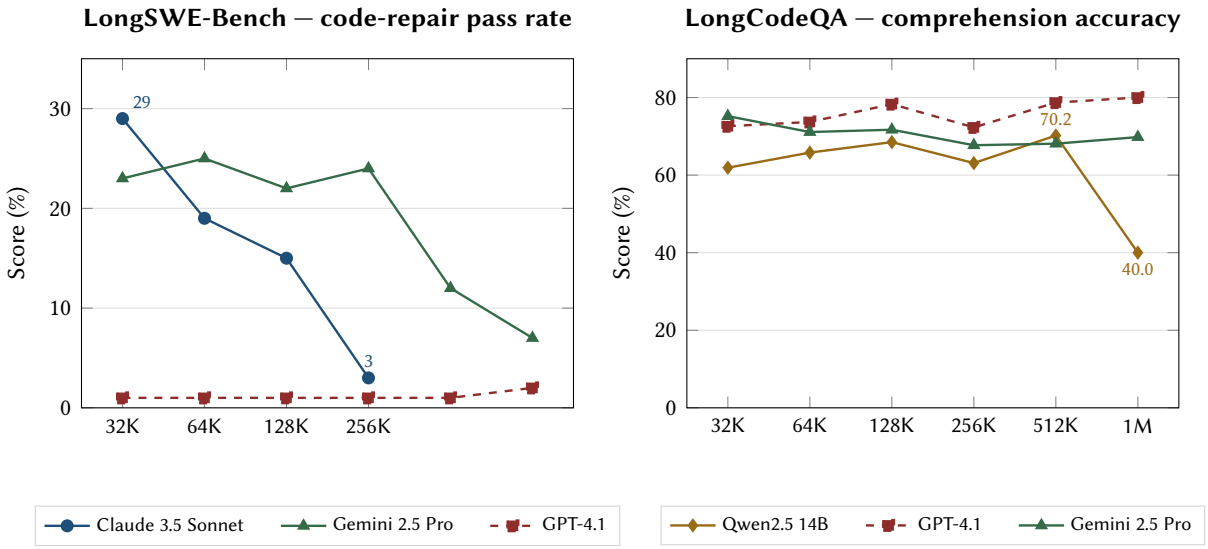

Figure 3: LongCodeBench reported per task, not collapsed into a single narrative. **Left:** bug-fix pass rate on real GitHub issues with executable patch grading. Claude 3.5 Sonnet falls from 29% at 32K to 3% at 256K and was not evaluated at 512K or 1M. Gemini 2.5 Pro holds 22–25% through 256K before its own cliff at 512K/1M. GPT-4.1 sits near 1% across all brackets. Qwen2.5, Llama 3.1, and Llama 4 Scout scored 0% at every bracket and are omitted. **Right:** multiple-choice comprehension. Qwen2.5 peaks at 70.2% at 512K and collapses to 40% at 1M, while GPT-4.1 and Gemini 2.5 Pro show no comparable cliff. The figure exists to argue calibration per model and per task, not a universal context-length threshold (Rando et al. 2025).

Three observations follow that single context degradation is not enough to characterize. First, the cliff is *not* monotonic across models: Gemini 2.5 Pro on LongSWE-Bench is the clean counterexample, and GPT-4.1 on LongCodeQA actively improves with more context. Second, the cliff is *not* the same across tasks: the ranking that picks the best 1M-token comprehender and the ranking that picks the best 256K-token bug-fixer are different rankings. Third, the often-quoted "Claude collapses from 29% to 3% at 1M" headline silently extrapolates beyond the published 256K bracket; the honest version is 32K to 256K. A router that picks a single "effective length" for all models and all tasks is discarding the part of the result that has operational consequences.

Figure 4 extends the same data to the full grid of ten models across six brackets and two tasks. The bug-fix grid is mostly red — four models score zero at every bracket; the comprehension grid is mostly green for GPT-4.1, Gemini 2.5 Pro, and Llama 4 Scout. The visual contrast is the operational point: the model that comprehends well at 1M and the model that repairs code well at 256K are not the same model.

LongSWE-Bench — code-repair pass rate (%)

Model	32K	64K	128K	256K	512K	1M
Claude 3.5 Sonnet	**29**	19	15	**3**	—	—
Gemini 2.5 Pro	23	25	22	24	12	7
GPT-4o	11	6	5	—	—	—
Gemini 2 Flash	10	6	7	3	2	2
Jamba 1.5 Large	3	1	1	0	—	—
GPT-4.1	1	1	1	1	1	2
Qwen2.5 14B	0	0	0	0	0	0
Llama 3.1 405B	0	1	0	—	—	—
Llama 4 Scout	0	0	0	0	0	0

LongCodeQA — comprehension accuracy (%)

Model	32K	64K	128K	256K	512K	1M
GPT-4.1	72.6	73.7	**78.3**	72.3	78.7	**80.0**
Gemini 2.5 Pro	75.2	71.1	71.7	67.7	68.1	69.8
Llama 4 Scout	66.4	73.7	70.7	63.1	78.7	76.0
Gemini 1.5 Pro	67.3	63.2	72.8	64.6	72.3	66.0
Claude 3.5 Sonnet	65.5	69.7	71.7	66.6	—	—
Jamba 1.5 Large	69.0	69.7	72.8	54.2	—	—
Qwen2.5 14B	61.9	65.8	68.5	63.1	**70.2**	**40.0**

Figure 4: Full LongCodeBench grid, ten models across six brackets. Green intensity tracks score; red intensity tracks floor performance; gray "—" means the model was not evaluated at that bracket. Two readings the side-by-side line chart in Figure 3 cannot deliver: **(top)** the bug-fix grid is mostly red — only Claude (at 32K) and Gemini 2.5 Pro (through 256K) clear single digits, and four models score zero at every bracket; **(bottom)** the comprehension grid is mostly green for GPT-4.1, Gemini 2.5 Pro, and Llama 4 Scout, with Qwen2.5's red cell at 1M visible as the genuine collapse. Same models, opposite stories. The router cannot reuse a model's comprehension reputation as a proxy for its repair reliability.

The newer context-rot literature sharpens the same point. Chroma's technical report evaluates 18 models and isolates cases where increasing input length alone makes outputs less reliable, including semantic needle variants, distractors, LongMemEval comparisons, and repeated-word copying tasks (Hong, Troynikov, and Huber 2025). NoLiMa extends needle-in-a-haystack beyond literal matching and reports severe drops when questions and needles have little lexical overlap, even when short contexts are nearly solved (Modarressi et al. 2025). AbsenceBench adds a different failure class: models that can retrieve present evidence may still fail to identify missing information, including omitted lines in GitHub pull-request diffs (Fu et al. 2025).

The defensible conclusion is not "1M always fails." The conclusion is that advertised context length is an upper bound, not an operating target. Routing budgets should be calibrated per model, task class, and representation, ideally by local evals or conservative defaults.

5.2 Retrieval can help or hurt

Z. Z. Wang et al. (2025) asks directly whether retrieval can augment code generation. Its answer is conditional: high-quality retrieved contexts improve some settings, especially weaker models and basic programming tasks, while retrievers still struggle when lexical overlap is limited and generators do not

always integrate extra context effectively. Asai et al. (2024) reaches a similar architectural lesson from a broader RAG setting: indiscriminate fixed retrieval can reduce versatility and produce unhelpful generations; retrieval should be adaptive.

Figure 5 crystallizes the empirical pattern. The same retrieval mechanism is a clear win in one quadrant (weaker models on basic tasks — CodeRAG-Bench reports +15.6 to +17.8 pp for StarCoder2-7B on HumanEval and MBPP) and a clear loss in the opposite quadrant (frontier reasoning models hit by semantic distractors). The operational consequence is that "do we retrieve?" has to be evaluated against *the agent's model strength and the task's complexity together,* not as a global default.

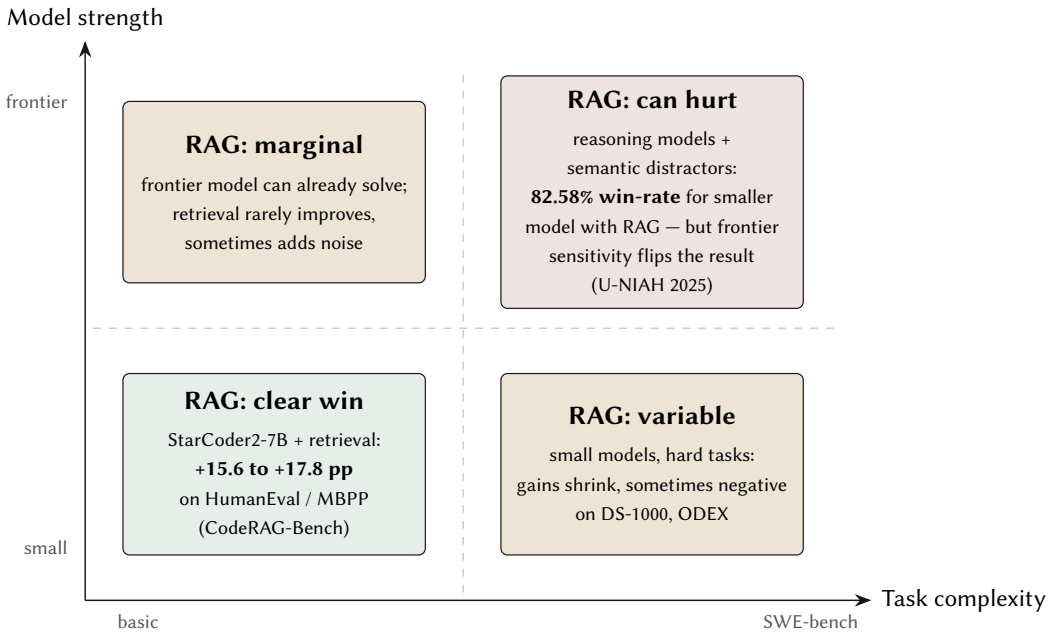

Figure 5: Whether retrieval helps depends on *both* model strength and task complexity. The bottom-left quadrant (CodeRAG-Bench: StarCoder2-7B +15–17 pp on HumanEval/MBPP) is where RAG is uncontroversial. The top-right quadrant (U-NIAH: frontier reasoning models distracted by semantic noise) is where adding retrieved context can actively reduce accuracy. A router should treat "do we retrieve?" as a conditional gate, not a default (Z. Z. Wang et al. 2025; Asai et al. 2024; U-NIAH 2025).

The router should not ask "RAG or not?" It should ask which retrieval substrate has the lowest expected miss, distractor, staleness, and latency cost for this task.

That means semantic and non-semantic search should be treated as different instruments, not rival slogans. BM25 remains a strong lexical baseline because exact terms, identifiers, and error strings often are the task (Robertson and Zaragoza 2009). Learned sparse retrieval such as SPLADE keeps lexical auditability while adding expansion terms (Formal, Piwowarski, and Clinchant 2021). Dense retrieval and text embeddings are useful when intent does not share surface form with the target, but benchmarks such as BEIR and MTEB show that retrieval quality is highly task- and distribution-dependent (Thakur et al. 2021; Muennighoff et al. 2023). CodeSearchNet made the natural-language-to-code gap explicit for semantic code search (Husain et al. 2019), while GraphCodeBERT shows why code structure and data flow matter beyond flat token embeddings (Guo et al. 2021). Late-interaction retrievers such as ColBERT occupy a useful middle ground: they retain token-level matching signals while allowing document representations to be precomputed

(Khattab and Zaharia 2020). ColBERTv2 and PLAID make the deployment point stronger by reducing late-interaction storage and serving cost without giving up the core matching advantage (Santhanam et al. 2022a; Santhanam et al. 2022b). Mixedbread's mxbai work is relevant as practitioner evidence for both dense embeddings and ColBERT-style reranking, and mgrep is a concrete coding-agent product claim that semantic search can reduce token usage relative to repeated grep loops while still preserving grep as the exact match fallback (Lee et al. 2024a; Lee et al. 2024b; Mixedbread 2026). Multi-function embedding systems such as BGE-M3 push in the same direction by supporting dense, sparse, and multi-vector retrieval modes inside one model family (Chen et al. 2024).

Recent code and technical-document retrieval benchmarks make the same lesson more concrete for software agents. FreshStack argues for benchmarks built from recent technical documentation, where off-the-shelf retrievers can lag far behind oracle context on niche stacks (Thakur et al. 2025). COIR separates code retrieval into multiple task families rather than treating "code search" as one problem (Li et al. 2024). CoSIL frames issue localization as graph-guided repository search rather than flat chunk retrieval (Jiang et al. 2025). For routing, these papers matter because they move the selector question from "semantic search or grep?" to "which retrieval substrate matches this task, authority, and failure mode?"

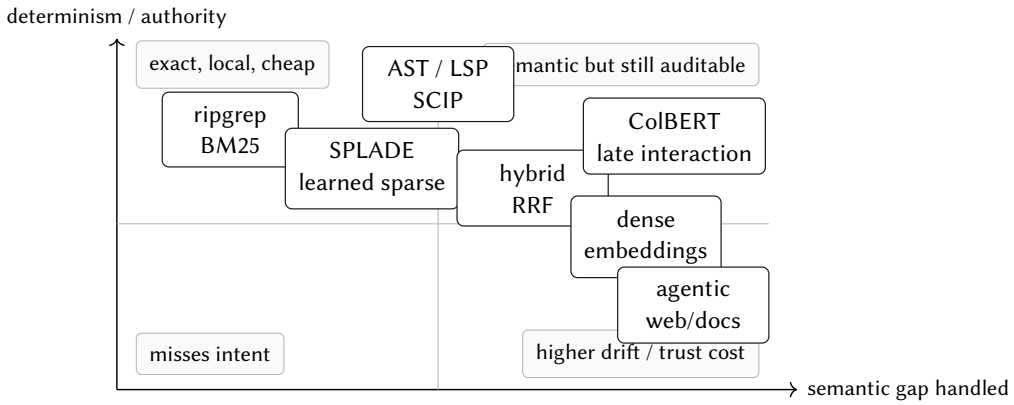

Figure 6: Semantic and non-semantic search are complements. Exact lexical search is cheap and deterministic; dense and late-interaction search bridge intent gaps; graph and symbol search preserve code authority; hybrid retrieval is often the pragmatic route.

Table 4: Retrieval substrates and their common failure modes.

Retrieval type	Typical failure mode
Lexical grep	Misses semantic matches and renamed concepts
Semantic vector search	Retrieves plausible but wrong chunks
Graph retrieval	Depends on graph quality and language support
AST chunking	Can miss runtime behavior and dynamic dispatch
Docs retrieval	Docs may be stale, incomplete, or for the wrong version
Agentic search	Slow, non-deterministic, and tool-call heavy
Hybrid search	More accurate but harder to debug and reproduce

5.3 Structure matters

Repository-level work consistently shows that flat text is a weak abstraction for code. F. Zhang et al. (2023), Shrivastava et al. (2023), Ding et al. (2024), Ding et al. (2023), Liang et al. (2024), Phan et al. (2025), and W. Liu et al. (2024) all point toward the same lesson: imports, definitions, call relationships, type information, symbols, and file structure provide routing signals that raw chunks do not. Y. (Sherry). Zhang et al. (2025) adds a newer preprint result for AST-aware chunking.

The careful implication is not "AST plus graph must always be the default." It is: for repository-scale retrieval, the prior should favor structural retrieval whenever the language ecosystem and latency budget permit it.

Figure 7 collects nine reported gains from six independent studies. The metric varies across studies (Recall@5, Pass@1, exact match, top-1 precision, identifier match), so the bars are not strictly comparable, but every published comparison points the same direction. Even the smallest cAST result (+2.7 percentage points on SWE-bench Pass@1) materially shifts real bug-fix rates; the larger relative gains (CoCoMIC +33.94%, RepoHyper +49%, REPOFUSE +50%, Code-Craft +82%) are large enough that the question for the router stops being "does structure help?" and becomes "which structural substrate fits this language and latency budget?"

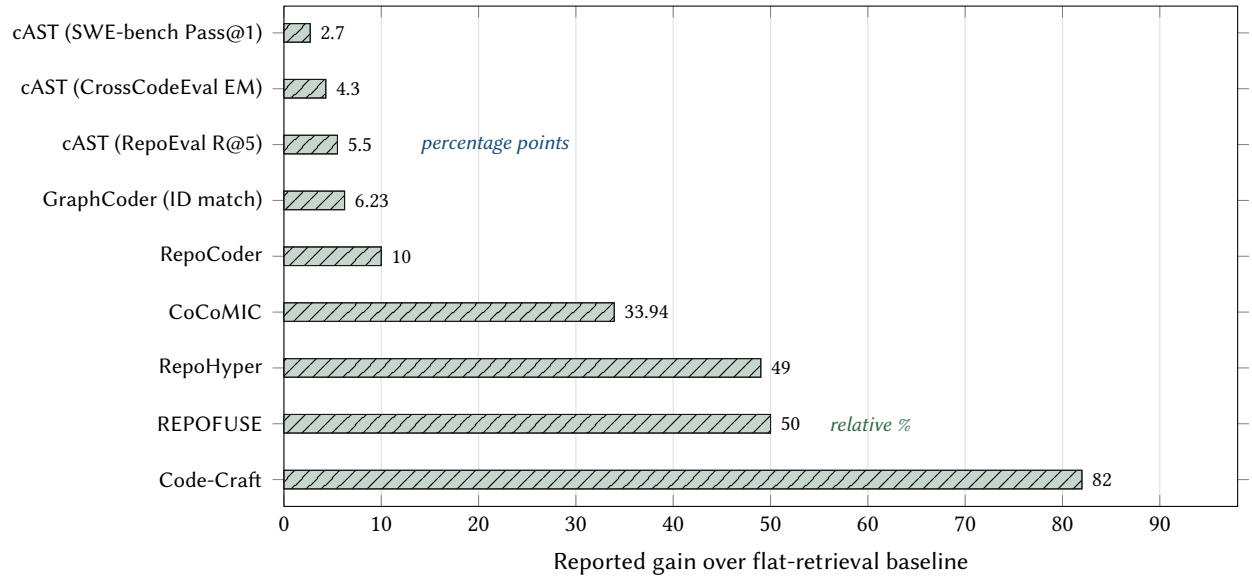

Figure 7: Nine reported results from six independent studies, all favoring structural or graph-aware retrieval over flat embedding retrieval. Top four bars (Code-Craft, REPOFUSE, RepoHyper, CoCoMIC) report *relative* % improvement; bottom five (RepoCoder, GraphCoder, three cAST settings) report *absolute* percentage-point gains, so the two groups should be read separately. The smallest cAST gain (+2.7 pp on SWE-bench Pass@1) is also the most adversarial benchmark; small pp gains on real bug-fix rates are large in practical terms. The router default for code retrieval should be structural; flat embeddings should be a fallback, not the front line (Code-Craft 2025; Phan et al. 2024; Liang et al. 2024; Ding et al. 2024; F. Zhang et al. 2023; W. Liu et al. 2024; Y. (Sherry). Zhang et al. 2025).

5.4 Wrong context has concrete harms

Context quality fails through recognizable mechanisms. Table 5 maps each failure mode to a router response.

Table 5: Failure modes that motivate routing.

Failure mode	Description	Router response
Attention dilution	Relevant tokens are present but weakly attended	Reduce bundle size; place high-value context early or late
Distractor interference	Plausible irrelevant context changes the reasoning path	Prune aggressively; prefer version-specific authority
Representation mismatch	Docs explain concepts but not usage patterns	Fetch examples, source, or tests
Version skew	Retrieved docs/source do not match installed package	Resolve through lockfile and provenance
Retrieval miss	Search fails to fetch the necessary file or symbol	Expand via graph, AST, LSP, or agentic search
Context staleness	Index reflects old repo state	Prefer live grep or incremental index
Trust contamination	External docs contain malicious or irrelevant instructions	Quote, sanitize, label provenance, sandbox tools
Cost blowup	Agent spends many turns searching	Precompute repo maps or local indexes

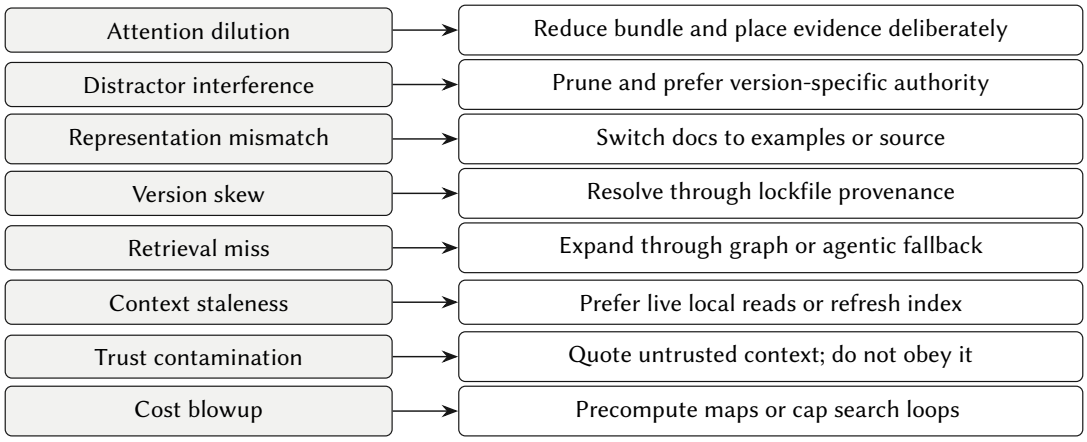

Figure 8: Wrong context has concrete failure modes. A router is useful only if each failure maps to an intervention: prune, switch representation, refresh, change authority, or stop searching.

Examples are easy to find in everyday agent work: the agent uses latest docs for an older installed package, imports a private symbol after reading source, acts on a StackOverflow answer for the wrong major version, searches a stale index while local uncommitted changes matter, follows instructions embedded in third-party docs, or receives a whole repo pack and overlooks the relevant file in the middle. These are not model failures alone. They are allocation failures.

5.5 Context changes during the agent loop

Context routing is online, not a one-time preprocessing step. Each phase of an agent loop wants different material.

Table 6: Task-phase routing.

Phase	Useful context
Orient	AGENTS.md, repo map, architecture summary, task brief
Locate	Code search, symbols, imports, stack traces, issue references
Plan	Relevant local files, docs, examples, package conventions
Edit	Exact files, types, tests, local wrappers, public API contracts
Verify	Test logs, CI output, type errors, runtime traces
Repair	Failing trace, package source, issue history, migration docs
Summarize	Decision log, changed files, unresolved risks, provenance

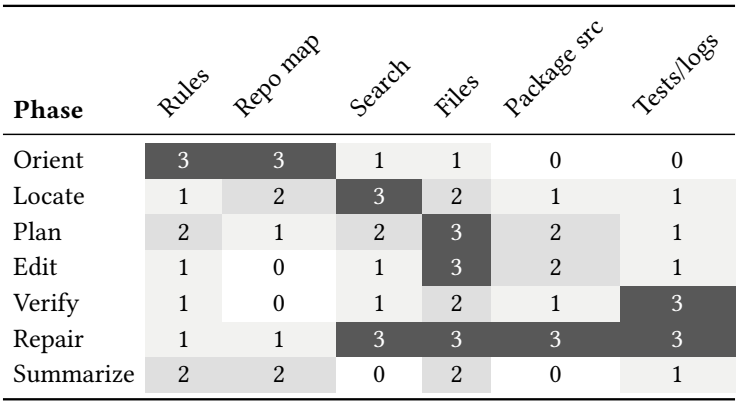

Figure 9: Task-phase context heatmap. Darker cells indicate higher expected usefulness. Context routing is an online policy: the right bundle changes as the agent orients, locates, edits, verifies, repairs, and summarizes.

This aligns with Anthropic's context-engineering framing: curation and maintenance of useful tokens happens throughout inference, not only at prompt construction time (Anthropic 2025a). Emerging OSS-focused context engineering work makes the same point at repository scale: agent configuration files, guidance documents, and tool manifests vary widely, which means context policy itself becomes part of the software architecture rather than an afterthought (Mohsenimofidi et al. 2025).

6 How to Think About Routing

When the decision matrix in §4 does not cover the case, the reader needs a framework for reasoning from scratch. This chapter is that framework: five orthogonal axes that any routing decision can be described along, followed by the mechanics of an explicit routing policy (inputs, scoring, authority, provenance, trust, explainability).

The axes are the same five used by every coding-agent stack on the market, whether or not their docs name them: where context lives (*placement*), what form it takes (*representation*), what chooses it (*selector*), when it loads (*timing*), and which constraint binds (*dominant constraint*). Naming them makes a fuzzy decision into a sequence of small, defensible decisions.

Before the axes, one principle holds across all of them: the operational goal is **smallest sufficient context**, not maximum context. Sufficiency means the agent can identify the relevant authority, produce a correct change, and verify it without materially misleading distractors. Smallest does not mean fewest tokens; it means lowest total cost across tokens, latency, privacy, staleness, trust, and distractor risk. Figure 10 shows the typical curve: utility climbs as relevant context is added, plateaus, then falls as the bundle grows past the model's effective attention.

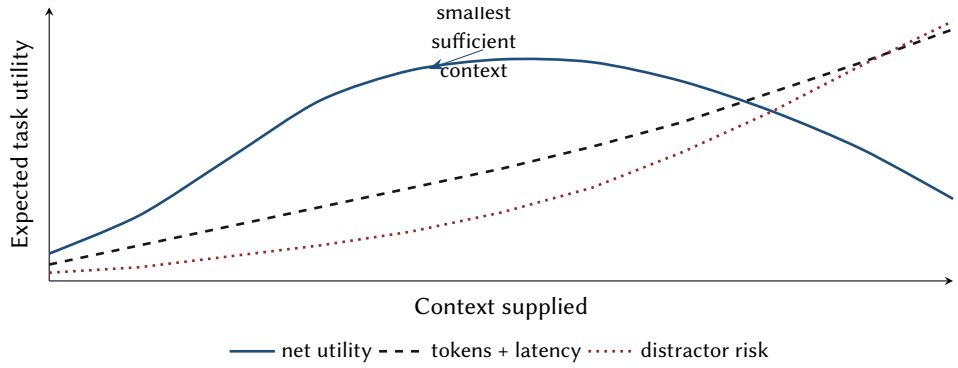

Figure 10: The router optimizes expected utility, not raw context size. More context helps until authority and relevance are outweighed by token, latency, staleness, trust, and distractor costs.

The five axes

The earlier "seven primitives" vocabulary is useful as shorthand, but it mixes dimensions. Vendoring, indexing, retrieving, packing, skills, annotations, and agentic reads are common bundles across several independent axes. A more useful taxonomy separates the axes first.

6.1 Axis 1: placement

Table 7: Where context lives.

Placement	Examples
In prompt	Pasted file, repo pack, selected snippets
On disk in workspace	Vendored source, cloned dependency, exported docs

Placement	Examples
Local cache	OpenSrc-style package cache, tool-managed snapshots
Local index	Sourcebot, Serena, local vector DB, SCIP/LSP graph
Remote service	Context7, Augment, Glean, hosted semantic search
Model or tool memory	Skills, summaries, persistent notes, agent memory

6.2 Axis 2: representation

Table 8: How context is represented.

Representation	Examples
Raw source	Vendored repository, fetched package source
Human docs	API docs, README, migration guides, changelogs
Symbol graph	SCIP, LSP, AST, call/reference graph
Embedding chunks	Vector search over files, docs, or issues
Summaries	Repo maps, architecture notes, skills, memory
Executable tools	MCP tools, scripts, local CLI commands
Instructions	AGENTS.md, CLAUDE.md, project rules

6.3 Axis 3: selector

Table 9: Who or what selects context.

Selector	Examples
Human	`@file`, explicit package, manual doc link
Static policy	allowlist routes `effect` to vendor
Heuristic router	import count, package size, privacy class
Learned retriever	embedding search, reranker
Graph retriever	symbol, dependency, call/reference expansion
Agentic search	`grep`/read/bash loop
Oracle/evaluator	benchmark-only upper bound

6.4 Axis 4: timing

Table 10: When context is loaded.

Timing	Examples
Always loaded	AGENTS.md, compact repo guidance
Session start	Repo map, recent decision log
Task start	Package docs, relevant tests, task issue

Timing	Examples
Just in time	Agent calls search, router fetches source
After failure	Fetch package source after stack trace points there
Compacted later	Summarize stale context for continuation

6.5 Axis 5: dominant constraints

Table 11: Constraints that dominate route choice.

Constraint	Examples
Token budget	Long-context degradation, prompt cost
Latency	Agentic search and remote retrieval delays
Privacy	Local-only code, no cloud index, no remote docs
Freshness	Local changes, docs drift, moving issues
Trust	MCP/tool/skill supply chain, prompt injection
Determinism	Reproducible context versus live retrieval
Version specificity	Lockfile-resolved package versions

The old primitives can now be expressed more precisely:

- **Vendoring** = workspace placement + raw source representation + static or heuristic selection + task-time loading + strong reproducibility + local privacy.
- **Indexing** = local or remote placement + symbol/embedding representation
  - learned or graph selector + low-latency query + staleness management.
- **Skills** = memory placement + instruction/tool representation + metadata-based selector + progressive disclosure + high token efficiency.
- **Repo pack** = prompt placement + raw or summarized representation + human or static selector + one-shot timing + weak freshness after creation.

For implementation, these axes become a route descriptor rather than a prose taxonomy.

Table 12: Route-descriptor axes for a context object.

Axis	Question	Example values
Object kind	What is being routed?	Package source, repo subtree, doc page, API spec, ticket, skill, live endpoint
Materialization	Where does it live for the task?	Committed, cached, packed, mounted, remote-only
Representation	In what form is it exposed?	Raw source, structural summary, prose doc, metadata envelope, search hit

Axis	Question	Example values
Access path	How does the agent reach it?	Direct read, local grep, local index, remote retrieval API, MCP tool
Freshness mode	How stable is it?	Pinned snapshot, refreshable cache, live mutable
Authority/trust	How much should the agent rely on it?	Canonical, upstream authoritative, derived, advisory, unverified
Privacy scope	What boundary constrains it?	Repo-safe, local-only, restricted, remote-call-required

Routing in practice

The five axes describe *where* a routing decision lives. The remainder of this chapter describes *how* an explicit policy actually selects a route: what inputs it consumes, how it scores candidates, how source authority resolves conflicts, how provenance travels with the bundle, how trust policy gates the choice, and how the route is explained to a human reviewer.

6.6 Inputs and outputs

A context router takes:

- task statement and task type;
- repository state, including local changes and lockfiles;
- knowledge object metadata;
- model and tool capabilities;
- privacy and trust policy;
- history of failures or user overrides.

It outputs:

- selected route and alternatives considered;
- context bundle or tool affordance;
- provenance record;
- trust class and intended use;
- explanation and override path;
- optional context-lock update.

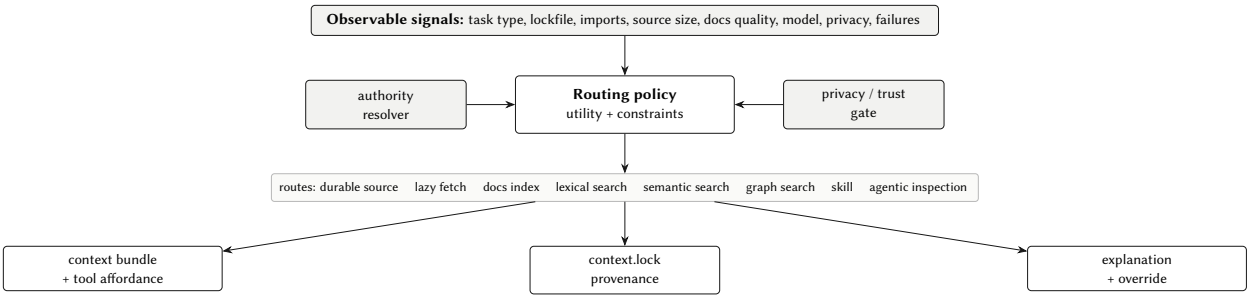

Figure 11: Context router architecture redrawn as a PDF-native diagram. The router does not merely retrieve content; it resolves authority, enforces trust policy, emits provenance, and explains the route.

6.7 Strategy scoring

The router can begin as a rule system, but the policy should expose scoring features so users can understand why a route was chosen.

Table 13: Scoring features across route families.

Feature	Vendoring	Lazy fetch	Docs	Index	Skill
Version fidelity	High if lockfile-resolved	High if resolved	Medium to high	Varies	Low unless versioned
Token efficiency	Medium	Medium	High	High	High
Agent navigability	High	High	Medium	Medium/high	Low/medium
Setup cost	Medium/high	Low	Low	High	Low
Privacy	High	High	Varies	Varies	High
Freshness	Pinned	Pinned	Fresh but may mismatch	Drift risk	Stable
Distractor risk	High if huge	Medium	Medium	Medium	Low
Best for	Deep idioms	Occasional inspection	Public API/migration	Large repos	Conventions/procedures

Figure 12 re-encodes the strategy choice as four cost axes (per-request latency, token cost per task, upfront/maintenance cost, and determinism) on a shared 1–10 scale. No strategy minimizes all four. Strategies on the left (vendor, cache, pack) pay an upfront and maintenance cost but recover it on latency, tokens, and determinism; strategies on the right (MCP fetch, agentic search) are cheap to set up and expensive per call, with agentic search the only strategy that is expensive on *both* latency and tokens while losing determinism. The router's job is to pick the strategy whose dominant cost the task can afford — not the strategy that is cheapest in the abstract.

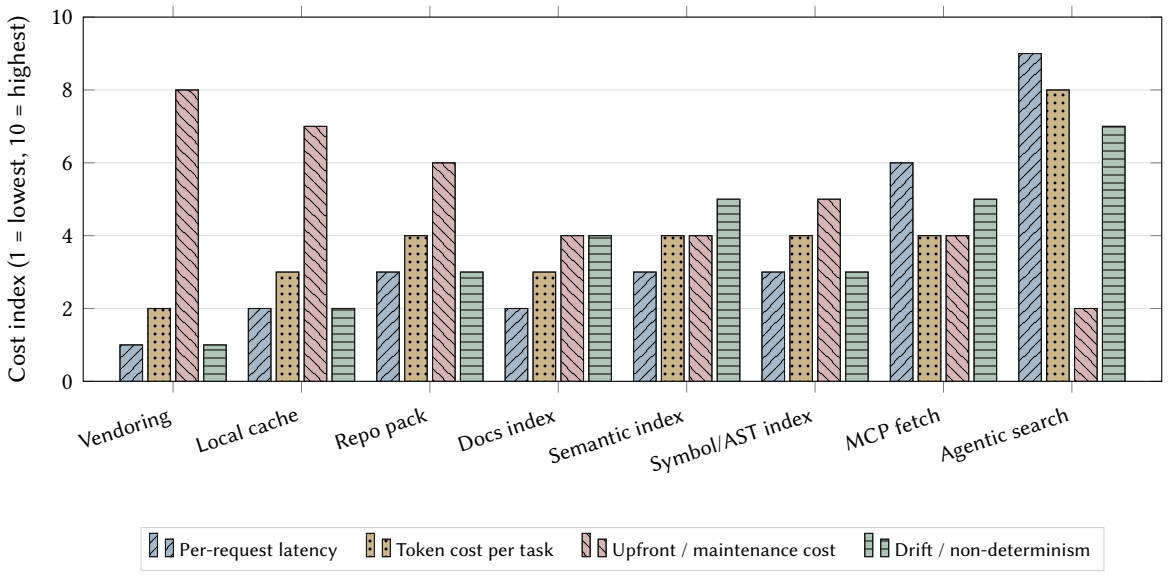

Figure 12: Four cost axes per routing strategy on a shared 1–10 scale. Every axis is oriented so *taller is worse* — low bars are strictly better than high bars. Strategies on the left (vendor, cache, pack) pay tall red bars for upfront and maintenance work but stay short on latency, tokens, and drift. Strategies on the right (MCP fetch, agentic search) skip the upfront work and pay per-call; agentic search is the only strategy with tall bars on *both* latency and tokens, and it also leads on drift because each turn is non-deterministic. No strategy minimizes all four; the router's job is to pick the strategy whose tallest bar the task can afford.

Default policy rules follow from the table:

- Vendor when package centrality is high, source size is manageable, privacy matters, and idioms matter.
- Lazy-fetch when a package is used occasionally but source-level inspection may be needed.
- Docs-first when the task targets public API, migration, conceptual usage, security guidance, or newly released features.
- Index when repository or package scale exceeds grep-friendly size, or cross-file relationships dominate.
- Skill when knowledge is procedural, stable, reusable, and safe to package.
- Agentic-search when local source is available and the model/tooling can inspect selectively.
- Pack when the agent lacks filesystem/search tools or the task is one-shot and offline.

6.8 Source authority and conflict resolution

A router should ask not only "where can I get information?" but "which source is authoritative for this question?"

Table 14: Source authority by question type.

Question	Best authority
What version is installed?	Lockfile and package manager metadata
What public API is intended?	Official docs and public type surface

Question	Best authority
What behavior occurs?	Version-matched source and tests
What pattern does this repo prefer?	Local codebase and AGENTS.md
What should the agent never do?	AGENTS.md, security policy, human instruction
What broke?	Failing test, log, trace, CI output
What changed between versions?	Changelog, release notes, migration guide
What symbols reference this?	LSP, SCIP, source index, grep

Conflict resolution should preserve provenance. A useful agent-facing message is not "use zod docs"; it is "this recommendation is based on local package source resolved from `bun.lock`, not latest web docs." That distinction prevents version skew and makes human override possible.

6.9 Provenance and context lockfiles

The router should emit context plus provenance. Example bundle metadata for source:

```yaml
context_bundle:
  source: npm
  package: effect
  version: 3.14.2
  route: vendor
  resolved_from: bun.lock
  fetched_at: 2026-05-13
  trust_class: local-readonly-source
  mutable: false
  intended_use: reference-only
```

For docs:

```yaml
context_bundle:
  source: context7
  package: next
  version_hint: "15.x"
  route: docs
  fetched_at: 2026-05-13
  trust_class: remote-docs
  warning: verify against lockfile and local source before editing
```

A `context.lock` file is the reproducibility bridge between research and implementation. It records the strategy, source URL, package version, commit, docs version, index version, skill version, trust class, generated summaries, timestamps, and override reasons.

```toml
[packages.effect]
strategy = "vendor"
version = "3.14.2"
```

```
source = "github:Effect-TS/effect"
commit = "abc123"
reason = "core-framework allowlist; imported in 42 files"
trust = "local-readonly"

[packages.zod]
strategy = "docs-first"
version = "4.1.0"
docs = "context7:zod@4.1"
fallback = "fetch-source"
reason = "public API task; source size low; docs available"
trust = "remote-docs"

[tools.mgrep]
enabled = false
reason = "privacy=strict-local"
```

Table 15: Core fields for a context lockfile.

Lockfile field	Purpose
`object_id`	Stable identity for the routed artifact
`kind`	Context-object class
`source_uri`	Upstream origin
`revision`	Commit, version, timestamp, or content hash
`authority`	Canonical, upstream, derived, advisory, or unverified
`materialization`	Committed, cache, mount, live, or pack
`representation`	Raw, summary, metadata, index, or executable tool
`privacy_class`	Repo-safe, local-only, restricted, or remote
`integrity`	Hash, signature, or provenance attestation
`phase_policy`	Preferred routes by task phase
`refresh_policy`	Pinned, manual refresh, TTL, or live
`transform_chain`	Export, normalization, and summarization steps

Provenance supports version fidelity, audit logs, privacy review, benchmark reproducibility, and human overrides.

6.10 Trust policy

Security is central to routing, not an appendix. Every route has a different attack surface.

Table 16: Route-specific security risks.

Route	Risk
Docs retrieval	Prompt injection, outdated examples, wrong version

Route	Risk
MCP server	Tool overpermission, data exfiltration, remote execution
Skills	Executable scripts, hidden instructions, supply-chain attacks
Vendored source	Malicious comments, enormous distractor surface
Repo pack	Secret exposure and uncontrolled redistribution
Cloud index	IP leakage and opaque retention
Agentic search	Command misuse and accidental broad reads
Generated summaries	Untraceable hallucinated compression

Recent MCP security research strengthens the authority distinction in this table. OX Security and Cloud Security Alliance both describe a 2026 MCP STDIO command-execution risk as systemic rather than a narrow bug (OX Security 2026; Cloud Security Alliance 2026). The practical router lesson is not "avoid MCP." It is that read-only context routes, local files, remote retrieval, and executable tool routes must be different trust classes.

A practical router should support policy modes such as:

- `strict-local`: no remote retrieval or cloud index;
- `local-executable-denied`: local files allowed, bundled scripts blocked;
- `remote-docs-allowed`: remote docs allowed but quoted as untrusted;
- `remote-code-denied`: no remote source fetch;
- `cloud-index-allowed`: managed indexes permitted;
- `scripts-require-confirmation`: skills and MCP tools cannot execute without confirmation;
- `untrusted-context-quoted`: external context is evidence, not instruction.

The ethical case for an open router follows from these requirements: transparent routing decisions, user override, local-first defaults, provenance logs, auditable lockfiles, no silent cloud upload, standards-based portability, and avoidance of vendor lock-in.

6.11 Explainability requirements

A useful routing explanation includes:

- selected strategy;
- alternatives considered;
- reason selected;
- risks;
- override command or config;
- provenance;
- expected cost.

Example:

> Routed `effect` to `vendor` because it is imported in 47 files, appears in the core-framework allowlist, source size is below 25 MB, and privacy policy is `strict-local`. Alternatives: docs-first rejected because the task requires idiomatic patterns; remote index rejected by privacy policy. Override with `ingraft route effect --strategy fetch`.

Explanations are not decoration. They are how users correct the policy.

6.12 Worked routing examples

Daily framework dependency. A project uses Effect in 80 files. The task is to implement a new service using local idioms. The router vendors Effect source read-only, emits AGENTS.md guidance, and tells the agent to inspect local examples before docs.

Rare utility package. The project uses `nanoid` in one file and the task is unrelated. The router does nothing. If the agent asks, it fetches source lazily.

API migration. A project upgrades Next.js from 14 to 15. The router prioritizes version-specific migration docs and changelog, then local usage search, then source fallback.

Large monorepo. The task touches one service in a 500K-file monorepo. The router avoids full packing, queries structural code search or SCIP/LSP, and passes only relevant files.

Privacy-restricted enterprise repo. Cloud retrieval is disabled. The router uses local source, local docs, local symbolic search, and blocks remote MCPs.

7 Standards and Interoperability

Standards matter because routing should not be trapped inside one agent vendor. The standards landscape is increasingly centered around AAIF projects such as MCP, goose, and AGENTS.md, alongside adjacent standards and conventions such as Agent Skills, ACP, A2A, llms.txt, and SCIP. It would be wrong to imply that all of these have one governance story.

The Linux Foundation announced the Agentic AI Foundation on December 9, 2025, with founding project contributions including Anthropic's Model Context Protocol, Block's goose, and OpenAI's AGENTS.md (Linux Foundation 2025). The same announcement describes MCP as a protocol for connecting models to tools, data, and applications, and AGENTS.md as a project-guidance standard for coding agents. Agent Skills has its own specification and progressive-disclosure model (Agent Skills Working Group 2025; Anthropic 2025b). ACP, A2A, llms.txt, and SCIP have separate origins and adoption paths (Zed Industries 2025; Google and Linux Foundation 2025; Howard 2024; Sourcegraph 2024).

Table 17: Standards and their routing role.

Standard or convention	Router role
AGENTS.md	Declares project-level routing policy and agent instructions
MCP	Exposes router tools to multiple agents
Agent Skills	Packages strategy-specific procedures with progressive disclosure

Standard or convention	Router role
ACP	Lets editors invoke agents that call the router
A2A	Lets agents coordinate or delegate context work
SCIP/LSP	Supplies graph and symbol context
llms.txt	Provides docs-discovery hints, low authority unless verified

The portable stack is therefore not "one standard to rule them all." It is: AGENTS.md for durable project policy, MCP for tool access, Agent Skills for procedural knowledge, and language/index standards for structural retrieval.

8 Where ingraft Fits

ingraft (Brunner 2026) is an open-source CLI for making repository context explicit for coding agents. Its implemented center of gravity is **durable source routing**: adding third-party packages and repositories into agent-ready reference directories with provenance, update tooling, and `AGENTS.md` guidance baked in. Its adjacent context commands route lighter cases to repo packs, lazy source fetches, and local search tools.

In the language of §2, ingraft implements the Strategy 1 durable-source route end-to-end, emits the Strategy 4 (instruct) glue that keeps durable source read-only for humans while remaining visible to agents, and wraps selected Strategy 3/5 routes when committing or linking source is too heavy. It does not try to be a full routing runtime; the seven strategies coexist in any non-trivial workflow, and many routes are best implemented by the tools surveyed in §3.

What ingraft does today

- Inspects the project's package manager state and lockfiles.
- Adds chosen upstream repositories as durable source routes under `vendor/`, including committed subtree routes with the same revision pin recorded in `.gitattributes` so PR diffs stay focused.
- Supports four durable-source modes: `subtree` (committed source), `submodule` (gitlink), `clone-ignore` (local ignored clone), and `cache-link` (ignored symlink to a shared cache checkout).
- Detects optional context tools and wraps lighter routes such as Repomix-style packs, OpenSrc-style lazy package source lookup, and local semantic search.
- Filters durable source routes to omit media, generated directories, archives, fixtures, and oversized files when configured.
- Generates `AGENTS.md` blocks listing durable source routes, the upstream URL and revision, and the rule that humans should not edit read-only reference files.
- Provides `list`, `add`, and `update` subcommands so the catalog stays maintainable.

When ingraft is the right pick

The fit matches Scenario 1 in §4.1: a project depends on a small set of idiom-heavy frameworks whose source is more useful to coding agents than their human-facing docs, where the team wants version-matched

local source available to every agent session without per-task fetches. Effect, Inertia.js, complex internal TypeScript frameworks, and "learn this library deeply" onboarding work are the natural domain.

When ingraft is the wrong pick

Vendoring is not the right strategy for migration tasks (Scenario 2), multi-repo enterprise refactors (Scenario 4), one-shot Q&A (Scenario 6), or security-sensitive codebases where the constraint is on-prem indexing rather than local source. For those, the tools and strategy combinations in §4 are better answers than ingraft.

Project structure

A workspace using ingraft typically looks like:

```
vendor/                          # committed, read-only reference
  effect/                         # subtree snapshot, version-pinned
  effect-smol/                    # another subtree

AGENTS.md                         # tells agents ' vendor/ is reference'
.gitattributes                    # marks vendor/ as vendored/generated
```

The full project layout and CLI reference are documented at https://ingraft.dev.

9 Conclusion

The 2026 context-engineering landscape is not a contest between "make source durable" and "use RAG." It is a catalog — seven strategies that solve different parts of the same problem — and a set of fit rules that say which combination wins for which task.

Three operational principles fall out of the guide:

1. **No strategy dominates.** The seven win on different task classes, different scales, and different constraints. A team that picks one and uses it for everything is leaving accuracy and tokens on the table.
2. **Combinations are normal.** Almost every real workflow uses two or three strategies together: vendor + tool-mediated + instruct; docs index + symbol index + tool-mediated; pre-built index + tool-mediated + repo map. The interesting design choices are about the combination, not the primary.
3. **Constraints choose the route before quality does.** Privacy, freshness, latency, and trust often eliminate strategies before the accuracy comparison even starts. Picking a route means knowing which constraints bind first.

The fit guide in §4 is the artefact most readers will return to. The strategies in §2 are the catalog that explains what each row means. The tools in §3 are the implementations to reach for. The evidence in §5 is the part to consult when the guide gets it wrong — which it will, because the field is moving and the May 2026 snapshot is dated by the time it is read.

Citation

If you reference this work in academic or industry writing:

> Brunner, G. *Context Engineering for AI Coding Agents: A Field Guide to Strategies, Tools, and Fit.* ingraft project, May 2026. https://ingraft.dev/whitepaper

Issues, corrections, and disagreements are welcome at github.com/gunta/ingraft/issues.

10 Appendix A: Representative Tool Snapshot, May 2026

This appendix preserves the tool-catalog value without letting the catalog drive the main argument. It is a representative snapshot, not a complete inventory.

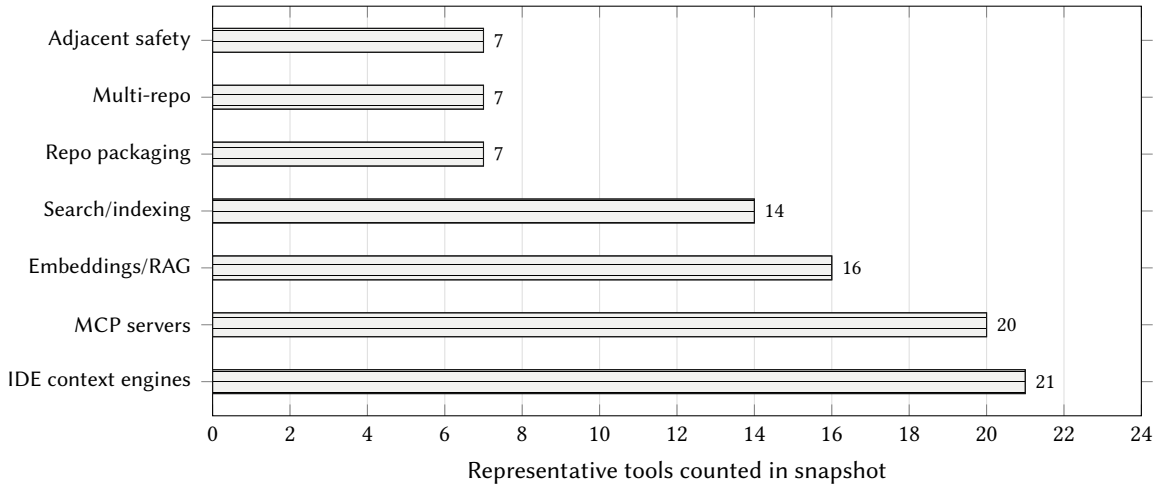

Figure 13: Tool-landscape fragmentation in the May 2026 snapshot. The distribution itself motivates routing: tools cluster around different context surfaces rather than one universal context strategy.

Table 18: Representative May 2026 tool snapshot.

Category	Representative tools	Routing lesson
IDE-native context engines	Cursor, Windsurf, GitHub Copilot, Augment, JetBrains AI/Junie, Cline, Zed, Roo Code, Continue, opencode, aider, Sourcegraph Amp, Amazon Q, Devin, OpenHands, Factory Droid, Kiro, Google Jules, Warp Agent Mode, Codebuff, Plandex	IDEs mix index, manual mention, agentic search, and rules; the router should expose which layer is active

Category	Representative tools	Routing lesson
Standalone code search/indexing	Sourcebot, Zoekt, Livegrep, Hound, ast-grep, Comby, Bloop, Greptile, Glean Code Search, Unblocked, Tabby, Serena MCP, CocoIndex, Indexify	Search wins when scale or cross-file structure overwhelms live grep
Repo packaging/snapshots	Repomix, gitingest, code2prompt, files-to-prompt, yek, ai-digest, gpt-repository-loader	Packs are useful one-shot snapshots but weak on freshness and position control
MCP context servers	Context7, filesystem MCP, GitHub MCP, GitLab MCP, DeepWiki MCP, fff, mgrep, Exa MCP, Tavily MCP, Perplexity MCP, MCPfinder, Glama, Smithery, mcp.so, Memory Bank, Mem0, Letta, Mnemos, Pieces LTM-2, OpenSrc	MCP turns context into tools; tool permissions and provenance become router concerns
Embeddings/RAG components	voyage-code-3, Jina code embeddings, Jina v2 base code, mxbai-embed-large, CodeBERT, GraphCodeBERT, UniXcoder, CodeT5+, StarCoder, CodeSage, ChromaDB, Qdrant, Pinecone, Weaviate, Milvus, pgvector	Embeddings are substrates, not policies; quality depends on selector, representation, and task
Multi-repo/org context	RepoSwarm, Backstage TechDocs, Roadie RAG, Glean, Sourcegraph, DeepWiki, Unblocked	Org context requires ACLs, provenance, and privacy-first routing
Adjacent safety/workflow	Qodo Merge, GitGuardian ggshield AI Hook, MCP-omnisearch, GPTScript, Jujutsu, agentjj, jj-skill	Context routing touches review, secrets, web search, shared prompts, and VCS workflows

11 Appendix B: Paper Matrix

Table 19: Academic and technical works mapped to router implications.

Work	Task	Context type	Main finding	Router implication	Evidence strength
N. F. Liu et al. (2024)	QA	Long context	Position matters	Prioritize placement and bundle size	Peer-reviewed journal
Hsieh et al. (2024)	Synthetic long-context	Multi-hop long context	Effective length varies by model/task	Calibrate per model	Peer-reviewed conference

Work	Task	Context type	Main finding	Router implication	Evidence strength
Rando et al. (2025)	Code QA and repair	Long repo context	Long context degrades unevenly on real code tasks	Avoid stuffing whole repos; evaluate by task	Preprint/benchmark
Hong et al. (2025)	Long-context reliability	Distractors, semantic needles, LongMemEval, repeated words	Increasing input length alone can make outputs less reliable	Treat context rot as a first-class routing cost	Technical report
Modarressi et al. (2025)	Long-context retrieval	Non-literal needles	Literal matching overstates long-context capability	Prefer semantic retrieval tests, not only lexical NIAH	Conference/preprint
Fu et al. (2025)	Missing-information detection	Omitted context	Models struggle to identify absences even when retrieval looks easy	Do not equate retrieval with comprehension	Conference/preprint
Z. Z. Wang et al. (2025)	Code generation	Retrieved docs/code	Retrieval helps unevenly and retrievers miss useful contexts	Adaptive retrieval, not fixed top-k	Peer-reviewed findings
Thakur et al. (2021)	Information retrieval	Heterogeneous retrieval tasks	No retriever dominates zero-shot across domains	Route by task distribution and retriever failure mode	Peer-reviewed benchmark
Khattab and Zaharia (2020)	Passage search	Late interaction	Token-level matching improves retrieval while allowing pre-computation	Use late interaction when dense vectors are too lossy	Peer-reviewed conference
Formal et al. (2021)	Information retrieval	Learned sparse retrieval	Sparse expansion bridges lexical auditability and semantic matching	Include sparse search in hybrid routing	Peer-reviewed conference
Husain et al. (2019)	Code search	Natural-language to code	Code search has a language gap distinct from generic text search	Evaluate code retrieval separately from text retrieval	Benchmark/preprint

Work	Task	Context type	Main finding	Router implication	Evidence strength
Li et al. (2024)	Code information retrieval	Multi-task code IR	Code retrieval decomposes into different task families	Do not tune one retriever for all code questions	Preprint/benchmark
Jiang et al. (2025)	Issue localization	Repository graph search	Graph-guided search can outperform flatter localization routes	Add graph search as an agentic fallback	Preprint
"U-NIAH" (2025)	Long-context/RAG	Synthetic needles/distractors	RAG helps some models and hurts others	Model-aware routing	Journal/preprint
F. Zhang et al. (2023)	Code completion	Iterative repo retrieval	Repository context improves completion	Expand context iteratively	Peer-reviewed conference
Ding et al. (2023)	Code completion	Cross-file context	Single-file context is insufficient	Use repo-aware retrieval	Peer-reviewed benchmark
Liang et al. (2024)	Code completion	Fused dual context	Context quality and latency trade off	Rank, truncate, and score context	Preprint
Phan et al. (2025)	Code completion	Semantic graph	Graph retrieval outperforms flat baselines	Prefer structural retrieval when possible	Workshop/preprint
Y. (Sherry). Zhang et al. (2025)	Code RAG	AST chunks	AST chunking improves over line chunks	Structural chunking	Preprint
Brown (2026)	Practitioner durable-source report	Effect source tree	Locally vendored source gives agents implementation structure, tests, and examples that docs and web fetches omit	Treat durable source as a candidate route for high-idiom, high-centrality packages	Practitioner/vendor evidence

Work	Task	Context type	Main finding	Router implication	Evidence strength
Gao (2026a)	Framework-agent evals	AGENTS.md docs index versus skills	Always-on compressed docs index outperformed on-demand skill use in their Next.js suite	Evaluate selector/timing, not only content quality	Vendor engineering eval
Shi et al. (2023)	Reasoning	Irrelevant context	Distractors reduce performance	Penalize distractor risk	Peer-reviewed conference

References

Agent Skills Working Group. 2025. "Agent Skills Specification." https://agentskills.io/specification.

Anthropic. 2025a. "Effective Context Engineering for AI Agents." September 29. https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents.

Anthropic. 2025b. "Equipping Agents for the Real World with Agent Skills." October 16. https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills.

Asai, Akari, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. "Self-RAG: Learning to Retrieve, Generate, and Critique Through Self-Reflection." *ICLR 2024*. https://arxiv.org/abs/2310.11511.

Baumann, Nick. 2025. "Why Cline Doesn't Index Your Codebase (and Why That's a Good Thing)." May 27. https://cline.bot/blog/why-cline-doesnt-index-your-codebase-and-why-thats-a-good-thing.

Brown, Maxwell. 2026. "The One Weird Git Trick That Makes Coding Agents More Effect-Ive." May 11. https://effect.website/blog/the-one-weird-git-trick-that-makes-coding-agents-more-effect-ive/.

Brunner, Gunther. 2026. "ingraft: The Context Router for AI Coding Agents." https://ingraft.dev/.

Cursor. 2026. "Securely Indexing Large Codebases." January 27. https://cursor.com/blog/secure-codebase-indexing.

Ding, Yangruibo et al. 2023. "CrossCodeEval: A Diverse and Multilingual Benchmark for Cross-File Code Completion." *NeurIPS 2023 Datasets and Benchmarks Track*. https://arxiv.org/abs/2310.11248.

Ding, Yangruibo, Zijian Wang, Wasi Uddin Ahmad, et al. 2024. "CoCoMIC: Code Completion by Jointly Modeling in-File and Cross-File Context." *Proceedings of LREC-COLING 2024*. https://arxiv.org/abs/2212.10007.

Gao, Jude. 2026a. "AGENTS.md Outperforms Skills in Our Agent Evals." January 27. https://vercel.com/blog/agents-md-outperforms-skills-in-our-agent-evals.

Gao, Jude. 2026b. "Feat(next-Codemod): Add Agents-Md Command for AI Coding Agents." January 24. https://github.com/vercel/next.js/pull/88961.

Gauthier, Paul. 2023. "Building a Better Repository Map with Tree-Sitter." October 22. https://aider.chat/2023/10/22/repomap.html.

Google, and Linux Foundation. 2025. "Agent-to-Agent Protocol (A2A)." https://a2a-protocol.org/latest/.

Howard, Jeremy. 2024. "The llms.txt Specification." September 3. https://llmstxt.org/.

Hsieh, Cheng-Ping et al. 2024. "RULER: What's the Real Context Size of Your Long-Context Language Models?" *COLM 2024*. https://arxiv.org/abs/2404.06654.

HumanLayer (Kyle). 2025. "Writing a Good CLAUDE.md." November 25. https://www.humanlayer.dev/blog/writing-a-good-claude-md.

Liang, Ming, Xiaoheng Xie, Gehao Zhang, Xunjin Zheng, Peng Di, et al. 2024. *REPOFUSE: Repository-Level Code Completion with Fused Dual Context*. https://arxiv.org/abs/2402.14323.

Linux Foundation. 2025. "Linux Foundation Announces the Formation of the Agentic AI Foundation." December. https://www.linuxfoundation.org/press/linux-foundation-announces-the-formation-of-the-agentic-ai-foundation.

Cloud Security Alliance. 2026. "MCP STDIO Design Flaw Enables Systemic AI Supply Chain RCE." April 23. https://labs.cloudsecurityalliance.org/research/csa-research-note-mcp-rce-design-vulnerability-20260423-csa/.

Liu, Nelson F., Kevin Lin, John Hewitt, et al. 2024. "Lost in the Middle: How Language Models Use Long Contexts." *Transactions of the Association for Computational Linguistics*. https://arxiv.org/abs/2307.03172.

Liu, Wei et al. 2024. "GraphCoder: Code Context Graph-Based Retrieval for Repository-Level Completion." *ASE 2024*. https://arxiv.org/abs/2406.07003.

Anthropic. 2026a. "Claude Code Changelog." May. https://code.claude.com/docs/en/changelog.

GitHub. 2026a. "GitHub Copilot CLI Is Now Generally Available." February 25. https://github.blog/changelog/2026-02-25-github-copilot-cli-is-now-generally-available/.

GitHub. 2026b. "Claude and Codex Now Available for Copilot Business & Pro Users." February 26. https://github.blog/changelog/2026-02-26-claude-and-codex-now-available-for-copilot-business-pro-users/.

Google. 2025. "Jules, Google's Asynchronous AI Coding Agent, Is Out of Public Beta." https://blog.google/innovation-and-ai/models-and-research/google-labs/jules-now-available/.

OpenAI. 2026a. "Work with Codex from Anywhere." May 14. https://openai.com/index/work-with-codex-

from-anywhere/.

OpenAI. 2026b. "Introducing the Codex App." February. https://openai.com/index/introducing-the-codex-app/.

OX Security. 2026. "The Mother of All AI Supply Chains: Critical Systemic Vulnerability at the Core of the MCP." April 15. https://www.ox.security/blog/the-mother-of-all-ai-supply-chains-critical-systemic-vulnerability-at-the-core-of-the-mcp/.

Phan, Huy N. et al. 2025. "RepoHyper: Search-Expand-Refine on Semantic Graphs for Repository-Level Code Completion." *FORGE 2025*. https://arxiv.org/abs/2403.06095.

Rando, Stefano, Luca Romani, Alessio Sampieri, et al. 2025. *LongCodeBench: Evaluating Coding LLMs at 1M Context Windows*. arXiv preprint. https://arxiv.org/abs/2505.07897.

Shi, Freda, Xinyun Chen, Kanishka Misra, et al. 2023. "Large Language Models Can Be Easily Distracted by Irrelevant Context." *ICML 2023*. https://arxiv.org/abs/2302.00093.

Shrivastava, Disha, Denis Kocetkov, Harm de Vries, Dzmitry Bahdanau, and Torsten Scholak. 2023. *RepoFusion: Training Code Models to Understand Your Repository*. arXiv preprint. https://arxiv.org/abs/2306.10998.

Sourcegraph. 2024. "SCIP — SourceCode Indexing Protocol." https://github.com/sourcegraph/scip.

Sourcegraph. 2025. "Why Sourcegraph and Amp Are Becoming Independent Companies." December. https://sourcegraph.com/blog/why-sourcegraph-and-amp-are-becoming-independent-companies.

"U-NIAH: Unified RAG and LLM Evaluation for Long Context Needle-in-a-Haystack." 2025. *ACM Transactions on Information Systems*. https://arxiv.org/abs/2503.00353.

Vercel. 2026. "next-evals-oss: Agent Evaluations for Next.js Coding Tasks." https://github.com/vercel/next-evals-oss.

Wang, Shawn (swyx), and Alessio Trotta. 2025. "Latent Space Podcast: Claude Code with Boris Cherny and Catherine Wu." May 7. https://www.latent.space/p/claude-code.

Wang, Zora Zhiruo et al. 2025. "CodeRAG-Bench: Can Retrieval Augment Code Generation?" *NAACL Findings 2025*. https://arxiv.org/abs/2406.14497.

Zed Industries. 2025. "Agent Client Protocol (ACP)." https://zed.dev/acp.

Zhang, Fengji, Bei Chen, Yue Zhang, et al. 2023. "RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation." *Proceedings of EMNLP 2023*. https://arxiv.org/abs/2303.12570.

Zhang, Yilin (Sherry) et al. 2025. *cAST: Enhancing Code RAG with Structural Chunking via Abstract Syntax Tree*. https://arxiv.org/abs/2506.15655.

Chen, Jianlv, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2024. *BGE M3-Embedding:*

Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. arXiv preprint. https://arxiv.org/abs/2402.03216.

Formal, Thibault, Benjamin Piwowarski, and Stéphane Clinchant. 2021. "SPLADE: Sparse Lexical and Expansion Model for First Stage Ranking." *SIGIR 2021.* https://arxiv.org/abs/2107.05720.

Fu, Harvey Yiyun, Aryan Shrivastava, Jared Moore, Peter West, Chenhao Tan, and Ari Holtzman. 2025. "AbsenceBench: Language Models Can't Tell What's Missing." *NeurIPS 2025 Datasets and Benchmarks Track.* https://arxiv.org/abs/2506.11440.

Hong, Kelly, Anton Troynikov, and Jeff Huber. 2025. "Context Rot: How Increasing Input Tokens Impacts LLM Performance." Chroma Technical Report. https://trychroma.com/research/context-rot.

Husain, Hamel, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. *CodeSearchNet Challenge: Evaluating the State of Semantic Code Search.* arXiv preprint. https://arxiv.org/abs/1909.09436.

Jiang, Zhonghao, Xiaoxue Ren, Meng Yan, Wei Jiang, Yong Li, and Zhongxin Liu. 2025. *CoSIL: Software Issue Localization via LLM-Driven Code Repository Graph Searching.* arXiv preprint. https://arxiv.org/abs/2503.22424.

Khattab, Omar, and Matei Zaharia. 2020. "ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT." *Proceedings of SIGIR 2020*, 39–48. https://doi.org/10.1145/3397271.3401075.

Lee, Sean, Aamir Shakir, Darius Koenig, and Julius Lipp. 2024a. "Open Source Strikes Bread – New Fluffy Embedding Model." Mixedbread. https://www.mixedbread.com/blog/mxbai-embed-large-v1.

Lee, Sean, Aamir Shakir, Darius Koenig, and Julius Lipp. 2024b. "ColBERTus Maximus – Introducing mxbai-colbert-large-v1." Mixedbread. https://www.mixedbread.com/blog/mxbai-colbert-large-v1.

Li, Xiangyang, Kuicai Dong, Yi Quan Lee, Wei Xia, Yichun Yin, Hao Zhang, Yong Liu, Yasheng Wang, and Ruiming Tang. 2024. *CoIR: A Comprehensive Benchmark for Code Information Retrieval Models.* arXiv preprint. https://arxiv.org/abs/2407.02883.

Mixedbread AI. 2026. "mgrep: Semantic Search for Agents." https://www.mgrep.dev/.

Modarressi, Ali, Hanieh Deilamsalehy, Franck Dernoncourt, Trung Bui, Ryan A. Rossi, Seunghyun Yoon, and Hinrich Schütze. 2025. "NoLiMa: Long-Context Evaluation Beyond Literal Matching." *International Conference on Machine Learning.* https://arxiv.org/abs/2502.05167.

Mohsenimofidi, Seyedmoein, Matthias Galster, Christoph Treude, and Sebastian Baltes. 2025. *Context Engineering for AI Agents in Open-Source Software.* arXiv preprint. https://arxiv.org/abs/2510.21413.

Muennighoff, Niklas, Nouamane Tazi, Loic Magne, and Nils Reimers. 2023. "MTEB: Massive Text Embedding Benchmark." *Proceedings of EACL 2023.* https://arxiv.org/abs/2210.07316.

Robertson, Stephen, and Hugo Zaragoza. 2009. "The Probabilistic Relevance Framework: BM25 and Beyond."

Foundations and Trends in Information Retrieval 3 (4): 333–389. https://doi.org/10.1561/1500000019.

Santhanam, Keshav, Omar Khattab, Jon Saad-Falcon, Christopher Potts, and Matei Zaharia. 2022a. "ColBERTv2: Effective and Efficient Retrieval via Lightweight Late Interaction." *NAACL 2022*. https://aclanthology.org/2022.naacl-main.272/.

Santhanam, Keshav, Omar Khattab, Christopher Potts, and Matei Zaharia. 2022b. *PLAID: An Efficient Engine for Late Interaction Retrieval*. arXiv preprint. https://arxiv.org/abs/2205.09707.

Thakur, Nandan, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. 2021. "BEIR: A Heterogeneous Benchmark for Zero-Shot Evaluation of Information Retrieval Models." *NeurIPS 2021 Datasets and Benchmarks Track*. https://openreview.net/forum?id=wCu6T5xFjeJ.

Thakur, Nandan, Jimmy Lin, Sam Havens, Michael Carbin, Omar Khattab, and Andrew Drozdov. 2025. "FreshStack: Building Realistic Benchmarks for Evaluating Retrieval on Technical Documents." *NeurIPS 2025 Datasets and Benchmarks Track*. https://arxiv.org/abs/2504.13128.